

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

Purpose =====

This note summarizes the current architecture discussion for connecting planetary parameters, stellar forcing, atmospheric disequilibrium chemistry, surface water availability, and possible hydrothermal vent environments.

The main design principle is:

The physical solver should use physical parameters and computed states. World labels such as Earth-like, super-Earth, mini-Neptune, ocean world, or Titan-like should be assigned only after the solver produces its result.

Relevant Code Paths

=====

Input parser: src/input_data/read_input.py
Chemistry input/result data: src/chemical_env/chem_env_data.py
Chemistry dispatcher: src/chemical_env/chemical_environment.py
Atmosphere solver driver: src/atmosphere_solver/atm_struct_driver.py
Planet data: src/planet_params/planetary_params.py
Stellar data: src/stellar_params/stellar_data.py
Black-body radiation: src/stellar_params/black_body_radiation.py
Stellar spectrum plot: src/utilities/plot_stellar_spectrum.py
Volcanic-rock water model: src/environment/volcanic_rocks/water_level.py
pH estimator: src/environment/pH_estimate.py
Atmosphere test runner: TEST_SCRIPTS/run_atmdyn_solver.sh
Titan test runner: TEST_SCRIPTS/run_titan.sh

Input Files and Modes

=====

Atmospheric disequilibrium input -----

The script

TEST_SCRIPTS/run_atmdyn_solver.sh

writes a full input.json into:

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

TESTS/ATMOSPHERE/input.json

The relevant chemistry mode is:

```
"exo_chemistry": {
  "mode": "layered_disequilibrium",
  "atomic_abundances": {
    "H": 1.0,
    "He": 0.1,
    "C": 1e-4,
    "O": 5e-3,
    "N": 1e-5
  }
}
```

The stellar input currently contains:

```
"stellar_data": {
  "spectral_class": "G2V",
  "star_temperature": { "value": 5778, "units": "K" },
  "star_radius": { "value": 6.957e8, "units": "m" }
}
```

The stellar radius is given in meters because Pint did not recognize solar_radius in the input. The value 6.957e8 m is the nominal solar radius.

Current Architecture

=====

Input parsing -----

read_input.py parses:

- planetary parameters into PlanetaryEnvironmentParams;
- stellar parameters into StellarParams;
- local environment parameters into local_env_data;
- chemistry input into ChemEnvInput.

The summaries should belong to their data objects. Therefore PlanetaryEnvironmentParams and StellarParams expose summary

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

methods, and read_input.py only calls those methods.

Chemistry dispatcher -----

SimulateChemEnv dispatches chemistry modes:

```
if mode == "local_equilibrium":
    run EasyChem
if mode == "layered_disequilibrium":
    run AtmosphSolver
```

The desired separation is:

- ChemEnvInput remains chemistry-oriented.
- Stellar data is needed by the atmospheric solver, not by local equilibrium chemistry.
- AtmosphSolver consumes the stellar object when computing radiation.

Stellar Black-Body Radiation

=====

The black-body spectral radiance is implemented in:

src/stellar_params/black_body_radiation.py

The function is:

```
blackbody_B_lambda(wavelength, temperature)
```

It returns Planck spectral radiance:

```
B_lambda(T) =
frac{2hc^2}{lambda^5}
frac{1}{exp(hc/(lambda k_B T)) - 1}
```

with units:

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

$W \text{ m}^{-3} \text{ sr}^{-1}$.

Stellar flux at the planet -----

In AtmosphSolver, the stellar intensity is evaluated over a wavelength grid:

```
self.wavelength_grid = Q_(np.linspace(100.0, 1000.0, 1000), "nanometer")
self.stellar_B_lambda = self.stellar_data.B_lambda(self.wavelength_grid)
```

The range (100-1000) nm covers UV, visible, and near-infrared. It is a good first range for a Sun-like star, but it does not include the full infrared.

The base spectral flux at orbital distance is:

$$F_0, \lambda = \Omega_{\text{star}} B_{\lambda}(T_{\text{star}}) \left(\frac{R_{\text{star}}}{r}\right)^2,$$

where the solid angle factor cancels the steradian in (B_{λ}). In code, this is represented by multiplying by a solid-angle quantity:

```
stellar_disk_solid_angle = Q_(np.pi, "steradian")
self.F0 = (
    stellar_disk_solid_angle
    * self.stellar_B_lambda
    * (self.stellar_data.radius / self.planet_data.orbital_distance) ** 2
).to("W / AU**2 / m")
```

The intended unit for F_0 is:

$W \text{ AU}^{-2} \text{ m}^{-1}$,

because it has been integrated over solid angle.

Atmospheric Solver Output

=====

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

The atmosphere solver should eventually produce a structured atmospheric result, including:

- wavelength grid and stellar forcing;
- atmospheric layers;
- pressure and temperature profile;
- chemistry per atmospheric layer;
- surface temperature and pressure;
- surface partial pressures, especially (p_H_2O);
- surface water phase and water availability.

The solver should be independent of labels such as ``Earth-like'' or ``ocean world''. Those labels are diagnostics assigned after the physical state is computed.

World Classification

=====

After the atmosphere dynamics is solved, world type can be assigned from final computed quantities:

- planet mass and radius;
- bulk density or inferred volatile envelope;
- surface temperature and pressure;
- atmospheric composition;
- water partial pressure;
- liquid/ice/vapor water state;
- estimated surface water coverage.

The classifier should not control the solver. Instead:

solver output -> classifier -> label

Example labels:

- dry rocky world;
- wet rocky world;
- Earth-like;
- super-Earth;
- water world;
- ice world;

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

- steam world;
- mini-Neptune;
- Titan-like methane world.

Surface Water and Volcanic Rocks

=====

The volcanic-rock water model is in:

```
src/environment/volcanic_rocks/water_level.py
```

The intended split is:

- the atmosphere solver computes surface atmospheric/surface water state;
- water_level.py maps that state into local volcanic-rock water availability.

At present, water_level.py estimates a water factor from atmospheric (H₂O) partial pressure:

```
derive_atmospheric_h2o_water_factor(...)
```

This is a downstream consumer. It should not solve the atmosphere.

pH Estimation =====

The pH estimator lives in:

```
src/environment/pH_estimate.py
```

Its purpose is to estimate water pH from chemistry if liquid water is available, including volcanic-rock settings and later hydrothermal settings.

The current first-order logic is:

- consume partial pressures from a ChemEnvResult;
- dissolve acid/base gases with Henry-law approximations;
- estimate acid/base contributions to ([H⁺]) or

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

```
([OH^-]);
```

```
- return a pHResult.
```

Current species include:

- (CO₂), acid;
- (NH₃), base;
- (H₂S), acid.

Aqueous/Ocean Chemistry Backend

=====

For ocean or hydrothermal chemistry, the suitable backend is not EasyChem. EasyChem is for gas-phase atmospheric equilibrium.

The practical aqueous/geochemical backend should be PHREEQC, likely through phreeqpy, or later Reaktoro if a heavier backend is needed.

The backend input should include:

- temperature;
- pressure if needed;
- pH guess;
- redox state, (pe) or (Eh);
- dissolved totals or species concentrations;
- gas partial pressures for gas-water exchange;
- minerals and phases for water-rock interaction.

The planned coupling is iterative:

- atmosphere solver computes surface chemistry;
- pH estimator gives an initial pH guess;
- aqueous backend computes consistent aqueous speciation and pH;
- updated aqueous chemistry feeds back into the environment model;
- repeat until pH and composition converge.

Hydrothermal Vent Model

=====

Hydrothermal vents should be modeled as a gradient between two reservoirs.

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

Ambient ocean reservoir -----

Inputs come from atmosphere/surface/ocean coupling:

- (T_ocean);
- (P_ocean);
- (pH_ocean);
- dissolved species;
- redox state;
- gas exchange with the atmosphere.

Vent reservoir -----

Inputs come from geophysics and water-rock interaction assumptions:

- (T_vent);
- (P_vent);
- (pH_vent);
- reduced species such as (H₂), (CH₄), (H₂S);
- metal and mineral chemistry such as Fe, Mg, Si, sulfides;
- rock/mineral phases;
- flow rate and water-rock ratio.

Gradients -----

The vent can be approximated as a one-dimensional reactive channel:

$$T(x) = T_{\text{ocean}} + (T_{\text{vent}} - T_{\text{ocean}}) \exp(-x/L_T),$$

$$\text{pH}(x) = \text{pH}_{\text{ocean}} + (\text{pH}_{\text{vent}} - \text{pH}_{\text{ocean}}) \exp(-x/L_{\text{pH}}),$$

$$C_i(x) = C_{i,\text{ocean}} + (C_{i,\text{vent}} - C_{i,\text{ocean}}) \exp(-x/L_i).$$

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

The crucial physical contrasts are:

- $(T_{\text{vent}} - T_{\text{ocean}})$, mostly geophysics;
- $(\text{pH}_{\text{vent}} - \text{pH}_{\text{ocean}})$, mostly composition and water-rock chemistry;
- redox contrast between reduced vent fluid and ambient ocean.

Titan Methane Pool Notes

=====

The Titan local environment should represent a methane pool, not water.

The pool should have:

- methane solvent data;
- a time-dependent methane level;
- a spatial pool profile function stored in local environment data.

The current profile function lives in:

src/environment/pool_spatial_profile.py

The plotting utility for this profile is:

src/utilities/plot_titan_pool_profile.py

Near-Term Implementation Order

=====

Recommended order:

- Finish the atmosphere solver structure: layers, stellar forcing, pressure and temperature profile.
- Produce a clear surface state from the atmosphere solver.
- Use `pH_estimate.py` to estimate pH when water is present.
- Feed atmosphere-derived water state into `volcanic_rocks/water_level.py`.
- Add an aqueous chemistry backend driver, likely PHREEQC.

Atmosphere, Surface Water, and Hydrothermal Environment Architecture

- Add hydrothermal vent inputs as ocean and vent reservoirs.
- Add a classifier that labels the final world type after solving.